SIMATS ENGINEERING

ASSESSMENT TOOL 3

SCENARIO-BASED

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1107

Course Outcome Covered: CO2: Design UML diagrams to represent system requirements and architecture. (BL3)

Assessment Tool Weightage: 30%

Questions: 6 **Total Marks:** 30

Scenarios Solved

Scenario No.	UML Diagram	System Modelled
1	Use Case Diagram	Smart Cinema Ticket Booking System
2	Class Diagram	Smart College Management System
3	Sequence Diagram	Online Railway Reservation System
4	Activity Diagram	Smart Insurance Claim Processing System
5	State Diagram	Smart Washing Machine System
6	Deployment Diagram	Smart Energy Management System

Student Details

Particular	Details
Name of the Student	M Nishanth
Register Number	192511154
Department / Year	B. Tech CSE / 2nd Year
Date of Submission	18-10-2026

Annexure A: Code of Conduct

SCENARIO-BASED

I _____**M Nishanth (192511154)** _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ___**M Nishanth**_____

Date: _____**18-10-2026**_____

1. Introduction and Approach

Each scenario describes a system whose design is unclear or incomplete. For every scenario the system is analysed, the key issues are identified, the suitable UML diagram is chosen, the design decisions are justified, and the final diagram is drawn. UML offers different views of a system: use case diagrams show requirements, class diagrams show structure, sequence, activity and state diagrams show behaviour, and deployment diagrams show the physical architecture (Booch, Rumbaugh, & Jacobson, 2005; Object Management Group, 2017).

1.1 Scenarios Solved

Scenario	UML Diagram	View of the System	Marks
1	Use Case Diagram	Requirements	5
2	Class Diagram	Static structure	5
3	Sequence Diagram	Interaction over time	5
4	Activity Diagram	Business workflow	5
5	State Diagram	Life cycle of an object	5
6	Deployment Diagram	Physical architecture	5
	Total		**30**

1.2 Method Followed for Every Scenario

- **Understanding of the scenario:** what the system does and what is stated.
- **Key issues:** what is unclear or missing in the current design, and how the design fixes it.
- **Application of concepts:** why this diagram type is used and which notation it needs.
- **Decision making:** each design choice is listed with its reason.
- **Clarity:** the analysis is given in tables, each diagram is on its own landscape page, and a final check confirms the diagram follows the UML rules.

2. Scenario 1: Use Case Diagram (Smart Cinema Ticket Booking System)

2.1 Understanding of the Scenario

A Smart Cinema Ticket Booking System lets customers search movies, select seats, book tickets, cancel bookings and pay online. Theater managers update movie schedules and seat availability, and administrators manage users and system settings. The scenario states that the interactions between the actors and the system functions are not clearly defined.

2.2 Key Issues Identified

Issue	Effect on the System	How the Design Resolves It
I1: The payment provider is not identified as an actor.	Online payment has no clear owner.	Payment Gateway added as an external actor.
I2: No login step exists.	Anyone could book or cancel for others.	Authenticate User added and included in booking and cancelling.
I3: Seat availability is not checked when a seat is selected.	Two customers could get the same seat.	Check Seat Availability included in Select Seats.
I4: A booking is not tied to payment or confirmation.	A booking may be unpaid or unconfirmed.	Make Online Payment and Send Booking Confirmation included in Book Tickets.
I5: Cancelling a paid booking has no refund step.	Customers lose money or staff handle refunds by hand.	Process Refund added as an extension of Cancel Booking.
I6: Payment methods and offers are not defined.	The payment function is too vague to build.	Pay by Card, Pay by UPI and Apply Discount Coupon added.

2.3 Application of Concepts

A **use case diagram** is used because the scenario is about requirements and who does what (Jacobson, Christerson, Jonsson, & Övergaard, 1992; Cockburn, 2001). **«include»** is used for behaviour that is always needed, **«extend»** for optional or conditional behaviour, and **generalization** for a specialised form of a general use case. Actors are outside the system boundary and use cases are inside it.

2.4 Analysis: Actors, Use Cases and Relationships

Actor	Type	Responsibility
Customer	Primary	Searches movies, books and cancels tickets, and pays.
Theater Manager	Primary (staff)	Updates movie schedules and seat availability.
Administrator	Primary (staff)	Manages users and system settings.
Payment Gateway	Secondary (external)	Authorizes online payments.

ID	Use Case	Actor	Source
UC1	Search Movies	Customer	Given
UC2	Select Seats	Customer	Given
UC3	Book Tickets	Customer	Given
UC4	Cancel Booking	Customer	Given
UC5	Make Online Payment	Customer, Gateway	Given
UC6	Update Movie Schedule	Theater Manager	Given
UC7	Update Seat Availability	Theater Manager	Given
UC8	Manage Users	Administrator	Given
UC9	Manage System Settings	Administrator	Given
UC10	Authenticate User	Customer	Added (I2)
UC11	Check Seat Availability	Customer	Added (I3)
UC12	Process Refund	Customer	Added (I5)
UC13	Send Booking Confirmation	Customer	Added (I4)
UC14	Apply Discount Coupon	Customer	Added (I6)
UC15	Pay by Card	Customer, Gateway	Added (I6)
UC16	Pay by UPI	Customer, Gateway	Added (I6)

From	Relationship	To	Condition or Reason
Book Tickets	«include»	Select Seats	A booking always needs seats.
Book Tickets	«include»	Make Online Payment	A booking must be paid.

From	Relationship	To	Condition or Reason
Book Tickets	«include»	Authenticate User	Only a logged-in customer can book.
Book Tickets	«include»	Send Booking Confirmation	Every booking is confirmed.
Cancel Booking	«include»	Authenticate User	Only the owner can cancel.
Select Seats	«include»	Check Seat Availability	Availability is always checked.
Process Refund	«extend»	Cancel Booking	[booking already paid]
Apply Discount Coupon	«extend»	Make Online Payment	[customer has a coupon]
Pay by Card	Generalization	Make Online Payment	Specialised payment method.
Pay by UPI	Generalization	Make Online Payment	Specialised payment method.

2.5 Design Decisions and Rationale

Design Decision	Rationale
Search Movies does not include Authenticate User.	Visitors should browse movies without logging in; login is needed only when money or bookings are involved.
Process Refund is an «extend», not an «include».	A refund happens only if the cancelled booking was paid, so it is conditional.
Apply Discount Coupon is an «extend» of payment.	It is optional, and payment works without it.
Payment Gateway is drawn outside the boundary.	It is an external system that the cinema system uses but does not control.
Theater Manager and Administrator are kept as separate actors.	Managers control cinema content; administrators control users and settings, so their rights differ.

2.6 Use Case Diagram

The Use Case Diagram is given as Figure 1 on the next page.

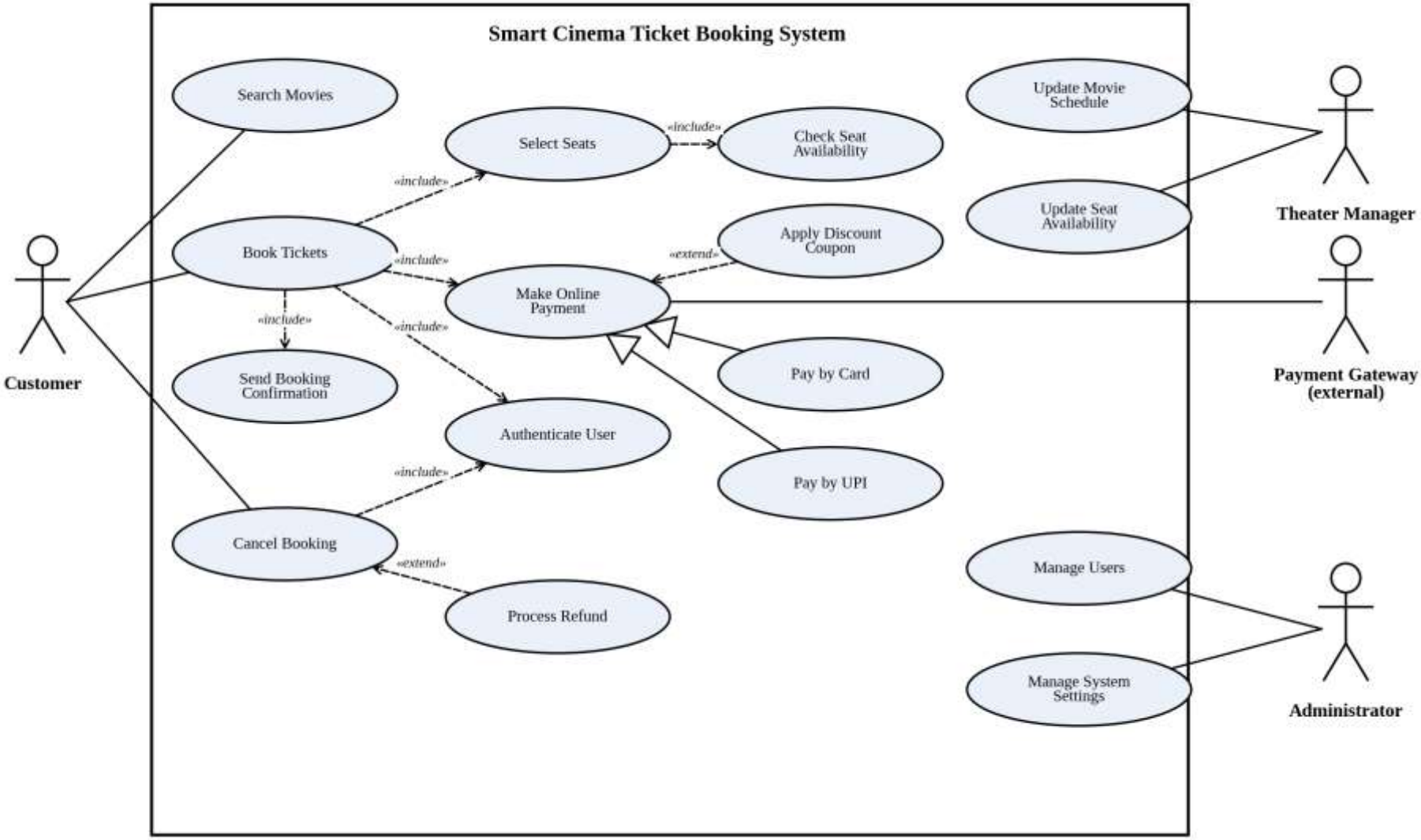


Figure 1: Use Case Diagram of the Smart Cinema Ticket Booking System

2.7 Final Check

Check	Result
All nine functions in the scenario appear as use cases, and all six issues (I1 to I6) are resolved.	Satisfied
Every actor is linked to at least one use case, and actors are outside the boundary.	Satisfied
«include» arrows run from base to included use case; «extend» arrows run from extension to base.	Satisfied
Generalization arrows point to the general use case (Make Online Payment).	Satisfied

3. Scenario 2: Class Diagram (Smart College Management System)

3.1 Understanding of the Scenario

The Smart College Management System has six named entities: Student, Faculty, Department, Course, Attendance and Examination. The design does not define their attributes, operations or relationships. They were worked out from these rules: a department employs faculty and offers courses; a student belongs to a department and enrolls in courses; a faculty member teaches courses; attendance is recorded for each student in each course; an examination belongs to a course and students appear in it; and students and faculty are both people.

3.2 Key Issues Identified

Issue	Effect on the System	How the Design Resolves It
I1: No attributes or operations are defined.	The classes cannot be implemented.	Attributes and methods defined for every class.
I2: Student and Faculty share personal data.	The same data would be written twice.	Abstract class Person added; both inherit from it.
I3: Ownership is unclear.	It is not known what is deleted with what.	Composition used for Department–Course and Course–Examination; aggregation for Department–Faculty.
I4: Students and courses are many-to-many.	A direct link cannot store attendance.	Enrolment shown as 0..* to 1..* and attendance made its own class.
I5: Multiplicities are missing.	Rules such as one department per student are lost.	Multiplicity specified on every association.

3.3 Application of Concepts

A **class diagram** is used because the scenario is about structure (Fowler, 2003). **Association** links objects of two classes, **aggregation** (hollow diamond) means a whole holds parts that can exist alone, **composition** (filled diamond) means the parts are deleted with the whole, and **inheritance** (hollow triangle) means a subclass is a kind of its parent. Attributes are private (-) and methods are public (+) for encapsulation.

3.4 Analysis: Classes, Relationships and Multiplicity

Class	Attributes	Methods
Person (abstract)	- personId: String - name: String - email: String - phone: String	+ getDetails(): String + updateContact(): void
Student	- rollNo: String - semester: int - admissionDate: Date	+ enroll(course): void + viewAttendance(): double + viewResult(): String
Faculty	- employeeId: String - designation: String - qualification: String	+ markAttendance(): void + conductExam(): void + uploadMarks(): void
Department	- deptId: String - name: String - hodName: String	+ addFaculty(f): void + addCourse(c): void
Course	- courseCode: String - title: String - credits: int	+ assignFaculty(f): void + getSyllabus(): String
Attendance	- attendanceId: String - date: Date - status: String	+ markAttendance(): void + getPercentage(): double
Examination	- examId: String - examDate: Date - maxMarks: int - type: String	+ scheduleExam(): void + publishResult(): void

From	Relationship	To	Multiplicity (first : second)
Student, Faculty	Inheritance	Person	–
Department	Aggregation (employs)	Faculty	1 : 1..*
Department	Composition (offers)	Course	1 : 1..*
Faculty	Association (teaches)	Course	1 : 1..*

From	Relationship	To	Multiplicity (first : second)
Student	Association (enrolls in)	Course	0..* : 1..*
Student	Association (admitted to)	Department	0..* : 1
Student	Association (has)	Attendance	1 : 0..*
Course	Association (recorded for)	Attendance	1 : 0..*
Course	Composition (has)	Examination	1 : 0..*
Student	Association (appears in)	Examination	0..* : 0..*

3.5 Design Decisions and Rationale

Design Decision	Rationale
Person is an abstract superclass.	Personal data and contact methods are written once and reused, which avoids duplication.
Department–Course is composition.	A course is part of one department's programme and has no meaning without it.
Department–Faculty is aggregation.	A faculty member can move to another department, so faculty exist independently of one department.
Course–Examination is composition.	An examination is set for a course and is deleted with it.
Attendance is a separate class.	Many attendance records exist for each student and course, so they need their own class.
Attributes are private.	Data can be changed only through methods, which keeps objects valid.

3.6 Class Diagram

The Class Diagram is given as Figure 2 on the next page.

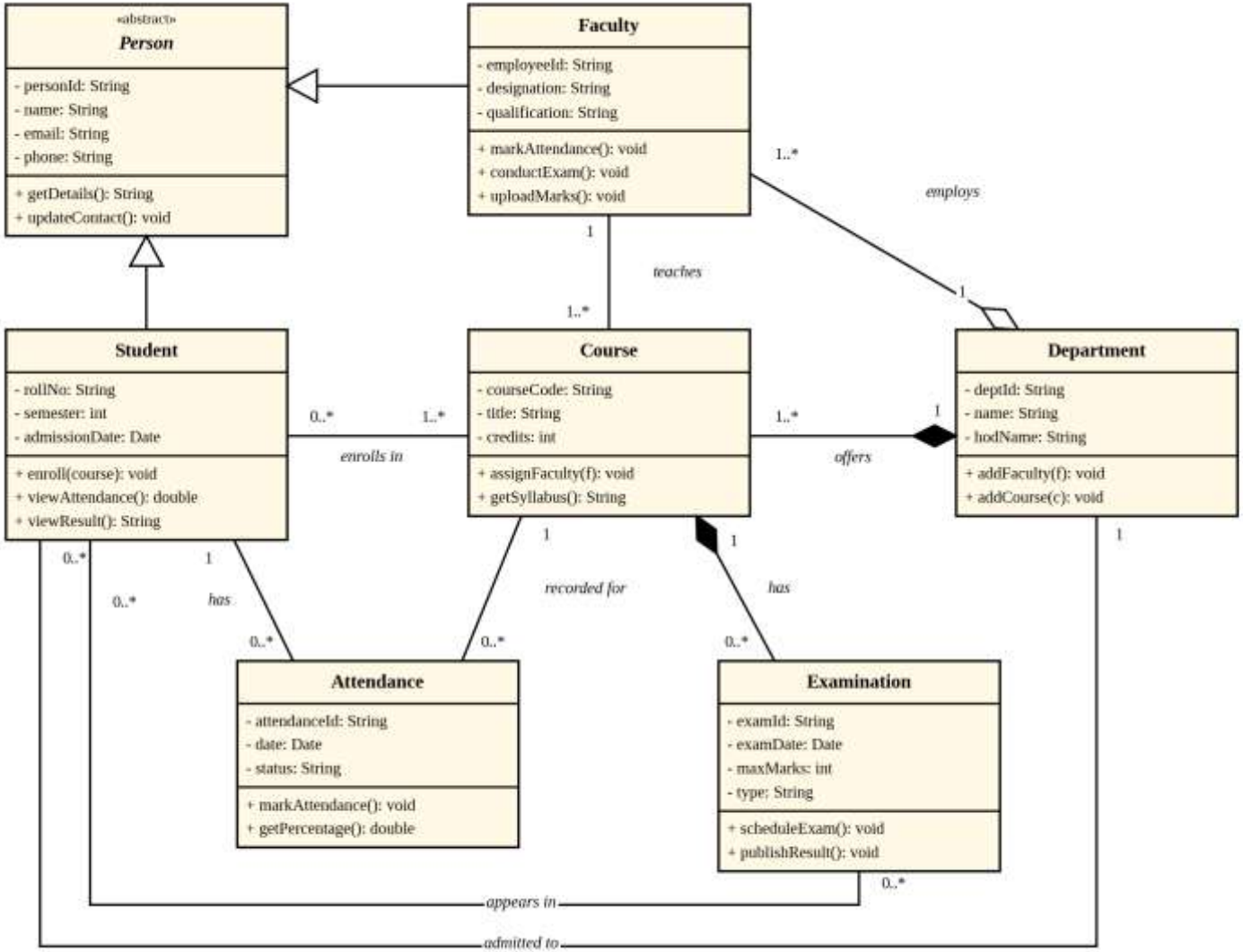


Figure 2: Class Diagram of the Smart College Management System

3.7 Final Check

Check	Result
All six named classes are present, and Person is added as an abstract parent.	Satisfied
All four relationship types (association, aggregation, composition, inheritance) are used.	Satisfied
Every association has a name and multiplicity at both ends.	Satisfied
Diamonds sit at the whole (Department, Course); the inheritance triangle points to Person.	Satisfied

4. Scenario 3: Sequence Diagram (Online Railway Reservation System)

4.1 Understanding of the Scenario

In the Online Railway Reservation System a passenger searches for trains, selects a seat, books a ticket, makes payment and receives an e-ticket. The participants named in the scenario are the passenger, the reservation system, the payment gateway and the database. The scenario states that the communication among them is incomplete.

4.2 Key Issues Identified

Issue	Effect on the System	How the Design Resolves It
I1: Train and seat data are not fetched from the database.	Nothing can be shown to the passenger.	Messages 2–3 and 6–7 added.
I2: A chosen seat is not held.	Two passengers could book the same seat.	holdSeat (message 10) added.
I3: The booking is not created before payment.	Payment has no booking to refer to.	createBooking (message 13) added.
I4: The payment result is not handled.	Success and failure are not separated.	alt fragment with two paths added.
I5: The booking is not confirmed after payment.	The database would keep an unconfirmed booking.	confirmBooking (message 18) added.
I6: The e-ticket step and failed-payment clean-up are missing.	No ticket is issued, and a failed payment keeps the seat locked.	generateETicket (20), eTicket (21) and releaseSeat (22) added.

4.3 Application of Concepts

A **sequence diagram** is used because the scenario is about the order of messages between objects (Fowler, 2003). Each object has a **lifeline** (dashed line), each period of work is an **activation bar**, calls are solid arrows, returns are dashed arrows, a **self message** loops back to the same object, and an **alt fragment** shows alternative paths chosen by a condition.

4.4 Analysis: Objects and Message Flow

Object	Role
Passenger	Actor who searches, selects, books and pays.
:ReservationSystem	Coordinates the whole booking and creates the e-ticket.
:PaymentGateway	External service that processes the payment.
:Database	Stores trains, seats, bookings and payments.

No.	From	To	Message	Type
1	Passenger	ReservationSystem	searchTrains(from, to, date)	Synchronous
2	ReservationSystem	Database	getTrains(from, to, date)	Synchronous
3	Database	ReservationSystem	trainList	Return
4	ReservationSystem	Passenger	displayTrains()	Return
5	Passenger	ReservationSystem	selectTrain(trainNo)	Synchronous
6	ReservationSystem	Database	getSeats(trainNo)	Synchronous
7	Database	ReservationSystem	seatMap	Return
8	ReservationSystem	Passenger	displaySeats()	Return
9	Passenger	ReservationSystem	selectSeat(seatNo)	Synchronous
10	ReservationSystem	Database	holdSeat(seatNo)	Synchronous
11	Database	ReservationSystem	seatHeld	Return
12	Passenger	ReservationSystem	bookTicket(passengerDetails)	Synchronous
13	ReservationSystem	Database	createBooking(details)	Synchronous
14	Database	ReservationSystem	bookingId	Return
15	Passenger	ReservationSystem	makePayment(amount)	Synchronous
16	ReservationSystem	PaymentGateway	processPayment(bookingId, amount)	Synchronous
17	PaymentGateway	ReservationSystem	paymentStatus	Return
18	ReservationSystem	Database	confirmBooking(bookingId)	Synchronous (if successful)

No.	From	To	Message	Type
19	Database	ReservationSystem	bookingConfirmed	Return (if successful)
20	ReservationSystem	ReservationSystem	generateETicket()	Self (if successful)
21	ReservationSystem	Passenger	eTicket	Return (if successful)
22	ReservationSystem	Database	releaseSeat(seatNo)	Synchronous (else)
23	ReservationSystem	Passenger	paymentFailed	Return (else)

Activation bars: the ReservationSystem is active while it handles each passenger request; the Database is active briefly for each read or write; the PaymentGateway is active only for messages 16 and 17. The passenger is an actor, so only a lifeline is drawn.

4.5 Design Decisions and Rationale

Design Decision	Rationale
The seat is held (message 10) before payment.	It stops another passenger from taking the same seat while payment is in progress.
The booking is created before payment.	The gateway needs a booking id so that the payment can be linked to the booking.
The ReservationSystem is the only object that talks to the gateway and the database.	The passenger and the gateway never touch stored data directly, which is safer and easier to control.
An alt fragment separates success from failure.	Each outcome needs different actions: confirm and issue a ticket, or release the seat and tell the passenger.
The e-ticket is a self message.	The reservation system creates the ticket itself; no other object is needed.

4.6 Sequence Diagram

The Sequence Diagram is given as Figure 3 on the next page.

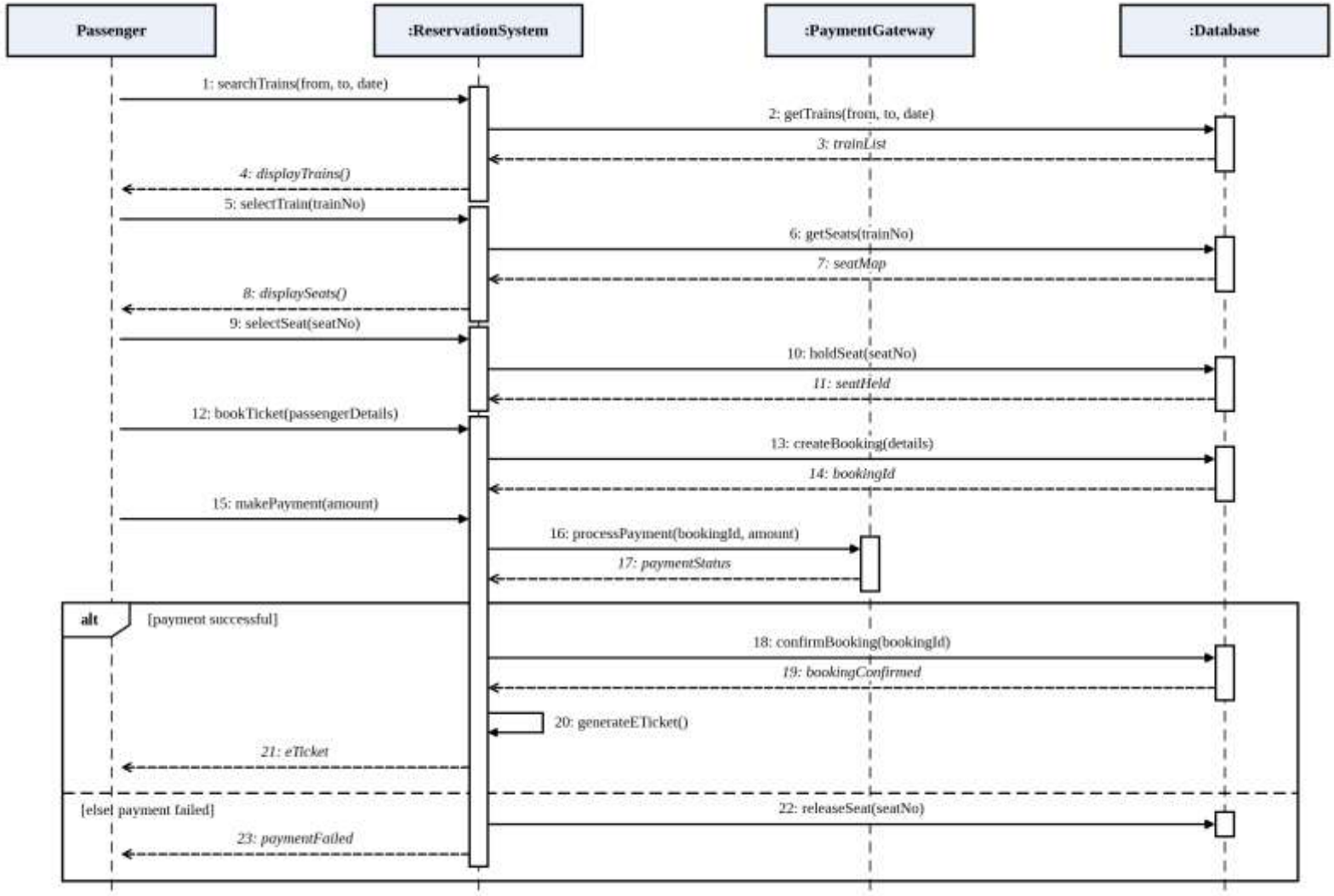


Figure 3: Sequence Diagram of the Online Railway Reservation System

4.7 Final Check

Check	Result
All four named participants have lifelines, and messages 1 to 23 run in time order.	Satisfied
Every synchronous call has a matching return, and activation bars start at a call and end at its reply.	Satisfied
The alt fragment has a guard on each path and covers messages 18 to 23.	Satisfied
All six issues (I1 to I6) are resolved by messages in the diagram.	Satisfied

5. Scenario 4: Activity Diagram (Smart Insurance Claim Processing System)

5.1 Understanding of the Scenario

The Smart Insurance Claim Processing System lets customers submit claims, upload supporting documents, have the claim verified, receive approval or rejection, and receive compensation. The scenario states that the workflow does not clearly show decisions or concurrent activities.

5.2 Key Issues Identified

Issue	Effect on the System	How the Design Resolves It
I1: Incomplete claims are not detected early.	Officers waste time on claims that cannot be processed.	Decision D1 checks the documents first and loops back for missing ones.
I2: Verification steps run one after another.	The customer waits longer than necessary.	Fork 1 runs policy, claim and fraud checks in parallel.
I3: There is no clear approve or reject decision.	The outcome of a claim is not controlled.	Decision D2 splits the flow into approval and rejection paths.
I4: Actions after approval are not organised.	Payment, notice and records may be forgotten or delayed.	Compensation is calculated, then Fork 2 runs payment, notice and records in parallel.
I5: Rejected customers are not informed.	The customer is left without an answer.	Notify Customer of Rejection added before the end.

5.3 Application of Concepts

An **activity diagram** is used because the scenario is about the flow of a business process (Booch, Rumbaugh, & Jacobson, 2005). A **decision node** (diamond) splits the flow by guard conditions in square brackets, a **fork** bar starts parallel activities, a **join** bar waits for all of them, a filled circle is the start, and a circle with a filled centre is an end.

5.4 Analysis: Activities, Decisions and Parallel Activities

ID	Activity	Performed By
A1	Submit Insurance Claim	Customer
A2	Upload Supporting Documents	Customer
A3	Request Missing Documents	System
A4	Verify Policy Status	System
A5	Verify Claim Details	Claims Officer
A6	Run Fraud Check	System
A7	Assess Claim	Claims Officer
A8	Approve Claim	Claims Officer
A9	Reject Claim	Claims Officer
A10	Notify Customer of Rejection	System
A11	Calculate Compensation	System
A12	Pay Compensation	System
A13	Send Approval Notification	System
A14	Update Claim Records	System

Node	Question	Guard	Next Activity
D1	Documents complete?	[complete]	Fork 1
D1	Documents complete?	[incomplete]	A3, then back to A2
D2	Claim valid?	[valid]	A8 Approve Claim
D2	Claim valid?	[invalid]	A9 Reject Claim

Fork / Join	Parallel Activities	Why They Are Independent
Fork 1 / Join 1	A4 Verify Policy Status, A5 Verify Claim Details, A6 Run Fraud Check	Each check uses different data and none needs another's result.
Fork 2 / Join 2	A12 Pay Compensation, A13 Send Approval Notification, A14 Update Claim Records	Once the amount is known, the three actions do not depend on each other.

5.5 Design Decisions and Rationale

Design Decision	Rationale
The completeness check D1 comes before verification.	Incomplete claims are returned at once, so no verification effort is wasted.
Three verification checks are placed in parallel.	The waiting time becomes that of the slowest check and not the sum of the three.
Calculate Compensation comes before the second fork.	Payment needs the amount, so this step is sequential and cannot be parallel.
Payment, notification and records are placed in parallel.	They are independent once the amount is known, so the customer is paid and informed sooner.
Rejection and completion have separate end nodes.	The two outcomes are different and are easy to tell apart.

5.6 Activity Diagram

The Activity Diagram is given as Figure 4 on the next page.

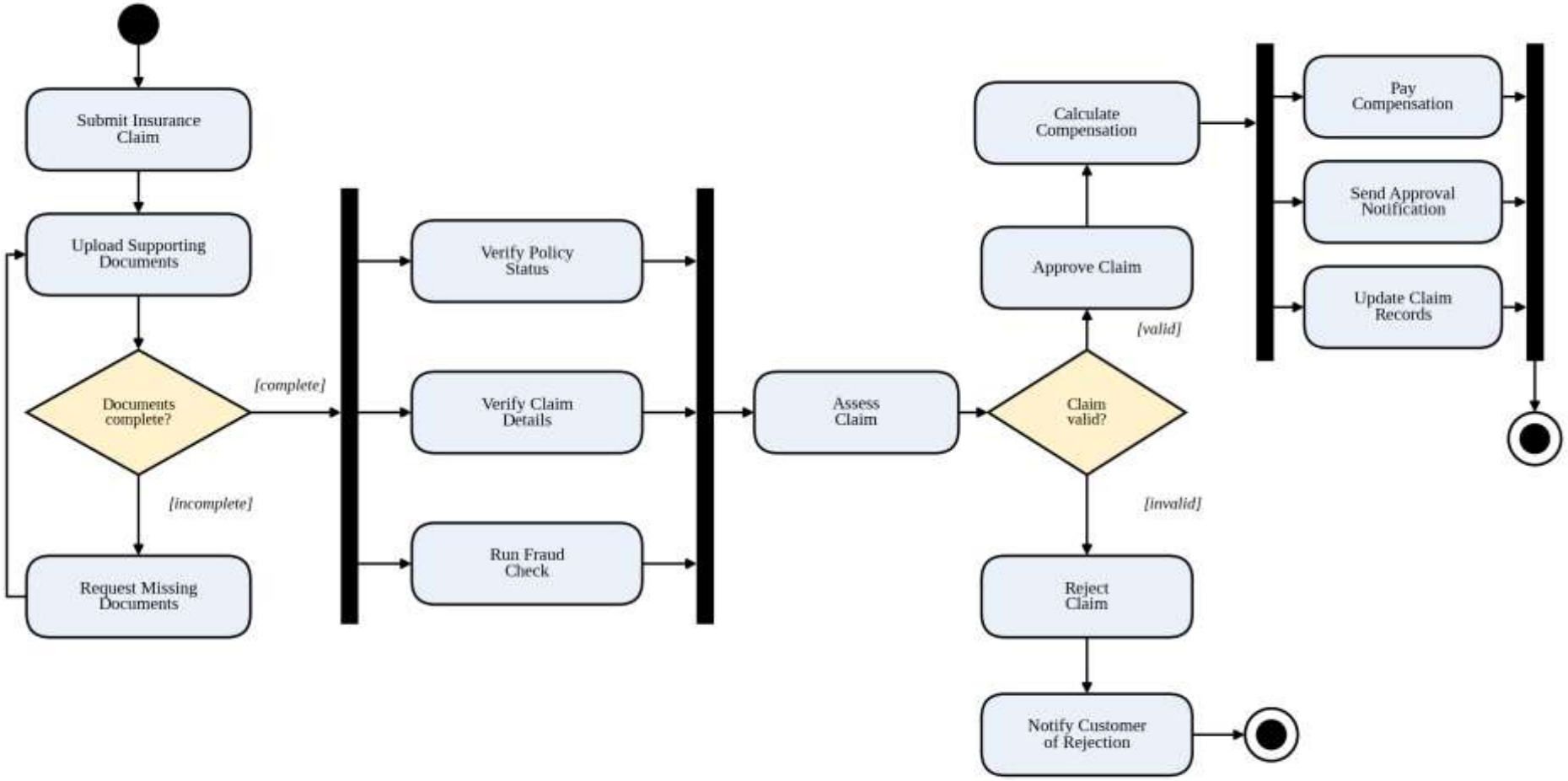


Figure 4: Activity Diagram of the Smart Insurance Claim Processing System

5.7 Final Check

Check	Result
The diagram has one initial node and two final nodes.	Satisfied
Every decision node has a guard on each outgoing flow, and the guards do not overlap.	Satisfied
Each fork has a matching join, and every activity A1 to A14 can be reached from the start.	Satisfied
All five issues (I1 to I5) are resolved.	Satisfied

6. Scenario 5: State Diagram (Smart Washing Machine System)

6.1 Understanding of the Scenario

A Smart Washing Machine System works through the states Idle, Water Filling, Washing, Rinsing, Spinning, Paused and Completed. The scenario states that the transitions between these states are not clearly specified. The machine runs a cycle of four stages in order, can be paused during the cycle, and returns to Idle when the clothes are removed.

6.2 Key Issues Identified

Issue	Effect on the System	How the Design Resolves It
I1: A pause can happen in any of four running states.	Four separate pause transitions would be needed.	The four stages are grouped in the composite state Cycle in Progress, which has one pause transition.
I2: After pausing, the machine must continue where it stopped.	Resuming from the start would waste water and time.	A history state (H) returns to the last active stage.
I3: The cycle must not start with the door open.	Water could leak and users could be hurt.	Guard [door closed] on the start transition.
I4: The end and reset of a cycle are unclear.	The machine may stay in Completed.	doorOpened resets Completed to Idle.
I5: A paused cycle cannot be abandoned, and the machine cannot be switched off.	The user is stuck.	cancelPressed returns Paused to Idle; powerOff leads to the final state.

6.3 Application of Concepts

A **state diagram** is used because the scenario is about how one object changes state in response to events (Harel, 1987; Object Management Group, 2017). A **state** is a condition the machine stays in, a **transition** is a change caused by an event (optionally with a guard in brackets and an action after a slash), a **composite state** groups several states, a **history state** (H) remembers the last active inner state, and the filled circle and the circle with a filled centre are the initial and final states.

6.4 Analysis: States and Transitions

State	Meaning	Action
Initial state	Power is connected.	–
Idle	Waiting for the user to start.	entry / powerStandby()
Water Filling	Drum is being filled with water.	entry / fillWater()
Washing	Drum agitates the clothes.	do / agitate()
Rinsing	Soap is rinsed out.	do / rinse()
Spinning	Water is removed by spinning.	do / spin()
Paused	Cycle is stopped by the user or by the door opening.	entry / stopMotor()
Completed	The cycle has finished.	entry / beep()
Final state	Machine is switched off.	–

No.	From	Event [Guard] / Action	To
1	Initial state	(power on)	Idle
2	Idle	startPressed [door closed]	Water Filling
3	Water Filling	waterFilled	Washing
4	Washing	washTimeUp	Rinsing
5	Rinsing	rinseTimeUp	Spinning
6	Spinning	spinTimeUp	Completed
7	Cycle in Progress (any stage)	pausePressed or doorOpened	Paused
8	Paused	resumePressed	History state (last stage)
9	Paused	cancelPressed	Idle
10	Completed	doorOpened / resetCycle()	Idle
11	Idle	powerOff	Final state

6.5 Design Decisions and Rationale

Design Decision	Rationale
Water Filling, Washing, Rinsing and Spinning are grouped in one composite state.	One pause transition covers all four stages, so the diagram is shorter and less error-prone.
A history state is used on resume.	The machine continues the stage it was in, instead of restarting the cycle.
The door is a guard on start and an event for pause.	Starting needs a closed door, and opening the door during a cycle must stop the motor for safety.
Paused can go to Idle through cancelPressed.	The user must be able to abandon a cycle, for example to add forgotten clothes and restart.
The final state is reached only from Idle.	The machine is switched off only when no cycle is running.

6.6 State Diagram

The State Diagram is given as Figure 5 on the next page.

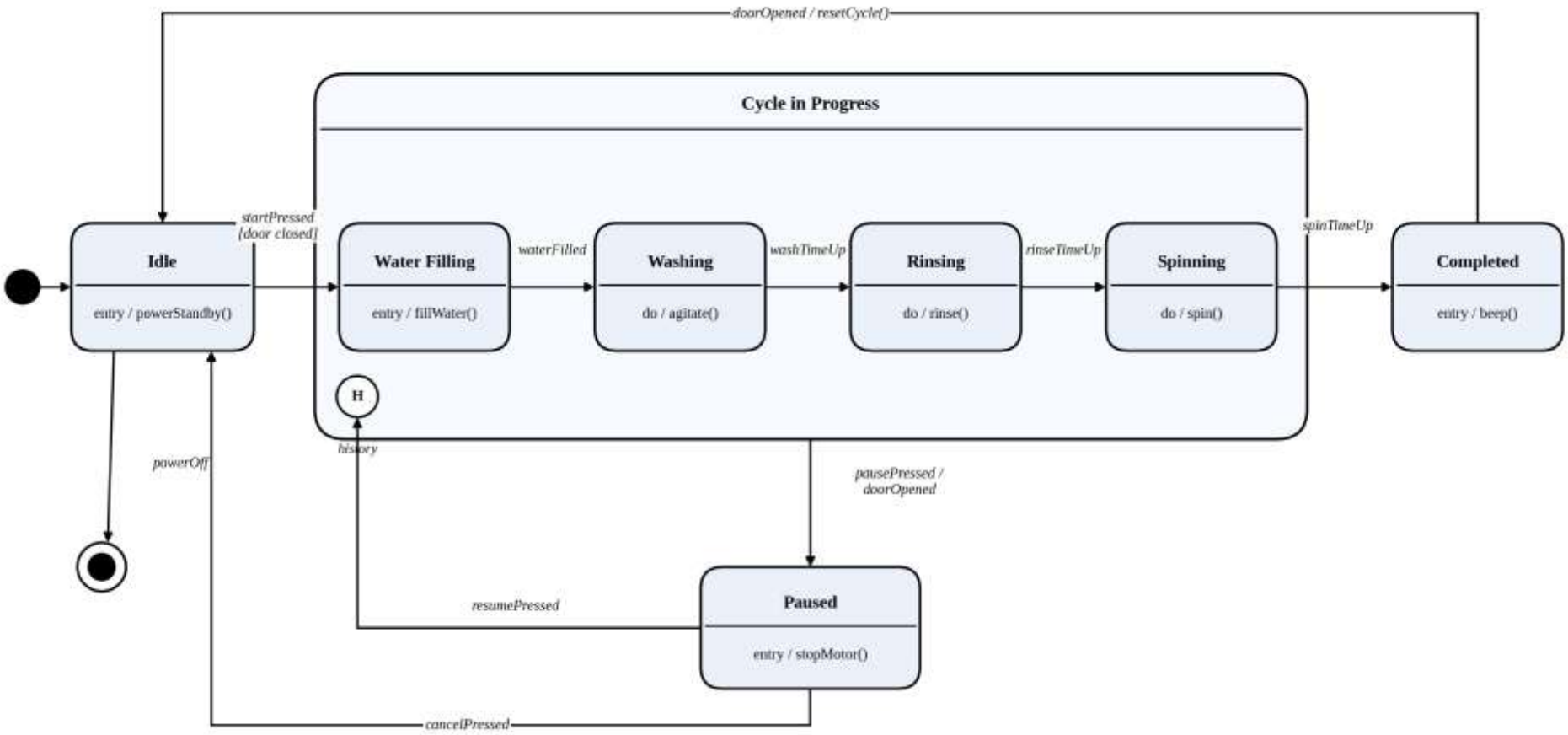


Figure 5: State Diagram of the Smart Washing Machine System

6.7 Final Check

Check	Result
All seven states given in the scenario are present, plus the composite state, history state, initial and final states.	Satisfied
Every transition has an event, with guards and actions where needed.	Satisfied
There is one initial state; the final state has no outgoing transition and can be reached.	Satisfied
Every state can be reached from the initial state and can be left (except the final state).	Satisfied

7. Scenario 6: Deployment Diagram (Smart Energy Management System)

7.1 Understanding of the Scenario

The Smart Energy Management System uses smart electricity meters, edge gateways, cloud servers, utility monitoring dashboards and mobile applications to monitor energy consumption and control power distribution. The scenario states that the deployment architecture does not clearly show the hardware nodes, the deployed software artifacts or the communication links.

Data flows **upward**: meters send readings to the gateway, and the gateway sends them to the cloud for storage and analysis. Control flows **downward**: when the cloud decides to change power distribution, a command goes to the gateway and then to a distribution control unit. A **Distribution Control Unit** was added as a node, because something in the field must carry out the switching.

7.2 Key Issues Identified

Issue	Effect on the System	How the Design Resolves It
I1: Hardware nodes are not clearly identified.	It is unclear what is installed where.	Six nodes defined with stereotypes.
I2: Software artifacts are not allocated.	It is unclear which software runs on which device.	Each artifact placed on one node.
I3: Communication links and protocols are not defined.	Devices cannot be connected or secured correctly.	Each link labelled with its protocol.
I4: The device that carries out power control is missing.	Control commands have no destination.	Distribution Control Unit added.
I5: Many meters would connect straight to the cloud.	High network load and weak security.	An edge gateway collects meter data and sends it over one secure link.

7.3 Application of Concepts

A **deployment diagram** is used because the scenario is about the physical architecture (Clements et al., 2010; Object Management Group, 2017). A **node** is a physical device or execution environment (a 3D box), an **artifact** is a file that runs on a node, and a **communication path** is a line between nodes labelled with its protocol.

7.4 Analysis: Nodes, Artifacts and Links

Node	Type	Artifact Deployed	Role
Smart Electricity Meter	Device	metering-firmware.bin	Measures energy use and sends readings.
Distribution Control Unit	Device (added)	switching-control.bin	Switches power distribution on command.
Edge Gateway	Device	edge-agent.jar	Collects meter data, applies local rules, relays commands.
Cloud Server	Execution environment	ingestion-service.war analytics-engine.jar load-control-service.jar rest-api.war energy-db.sql	Stores and analyses data, decides control actions, serves the apps.
Utility Monitoring Dashboard PC	Device	dashboard-ui.js	Shows consumption for the utility.
Consumer Mobile Device	Device	energy-app.apk	Shows consumption to the user.

Between	Protocol	Purpose
Smart Electricity Meter and Edge Gateway	RF mesh	Low-power wireless link for meter readings.
Distribution Control Unit and Edge Gateway	Modbus TCP	Reliable industrial link for switching commands.
Edge Gateway and Cloud Server	MQTT over TLS (4G / Ethernet)	Secure, lightweight two-way messaging.
Consumer Mobile Device and Cloud Server	HTTPS (REST)	Secure access to consumption data.
Utility Dashboard PC and Cloud Server	HTTPS (REST)	Secure access to the dashboard.

7.5 Design Decisions and Rationale

Design Decision	Rationale
An edge gateway sits between the field devices and the cloud.	It reduces network traffic, adds one secure connection point, and can keep simple control rules running if the cloud link fails.
MQTT over TLS is used from the gateway to the cloud.	MQTT suits small, frequent readings, and TLS protects the data in transit.
Modbus TCP is used for the control unit.	It is a widely used industrial protocol, so it works with common switching equipment.
Analysis and control decisions are deployed on the cloud server.	The cloud sees data from all meters, so it can balance the load across the whole network.
Dashboard and mobile app use HTTPS to the same REST API.	One interface serves both, and HTTPS protects what users see.

7.6 Deployment Diagram

The Deployment Diagram is given as Figure 6 on the next page.

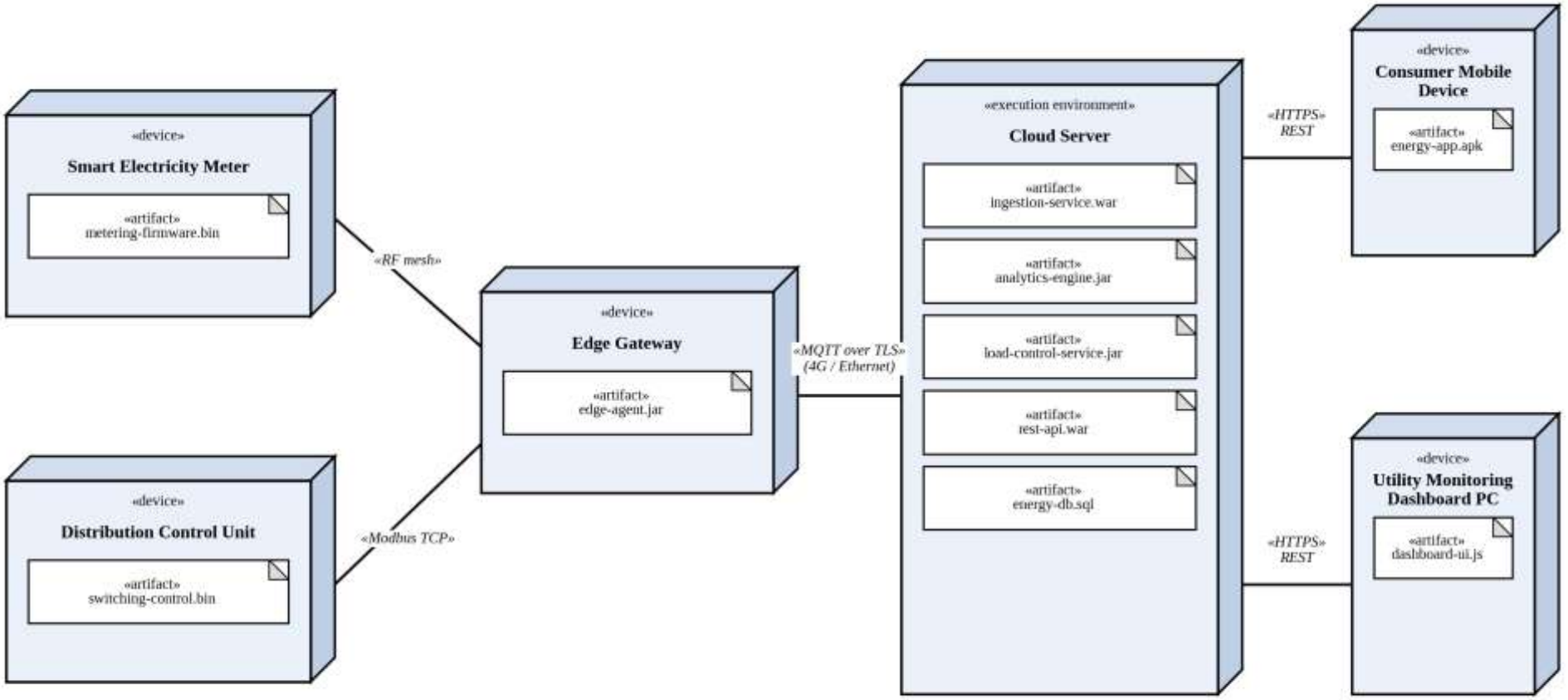


Figure 6: Deployment Diagram of the Smart Energy Management System

7.7 Final Check

Check	Result
All five elements named in the scenario (meters, edge gateways, cloud servers, dashboards, mobile applications) appear as nodes.	Satisfied
Every node shows its stereotype and every artifact is placed on the node where it runs.	Satisfied
Every communication path is labelled with a protocol, and no node is isolated.	Satisfied
All five issues (I1 to I5) are resolved.	Satisfied

8. Conclusion

All six scenarios were analysed and solved using the UML diagram that suits each. The table summarises the result.

Scenario	Diagram	Main Outcome
1	Use Case	4 actors and 16 use cases using include, extend and generalization.
2	Class	7 classes with attributes, methods, 4 relationship types and multiplicities.
3	Sequence	4 objects and 23 messages with an alt fragment and activation bars.
4	Activity	14 activities, 2 decision nodes and 2 parallel stages.
5	State	7 states plus a composite state, history state, initial and final states.
6	Deployment	6 nodes, 9 artifacts and 5 communication paths with protocols.

Each solution states what the scenario left unclear, makes a reasoned decision for every gap, and checks the result against the UML rules. Together the diagrams show requirements, structure, behaviour and physical architecture, which meets Course Outcome CO2.

9. Alignment with the Assessment Rubric

Rubric Criterion	Weightage	Where it is Addressed
Understanding of Scenario	20%	Sections 2.1 to 7.1 explain what each system does and what is unclear.
Identification of Key Issues	20%	Sections 2.2 to 7.2 list the issues, their effect and how the design resolves them.
Application of Concepts	25%	Sections 2.3 to 7.3 and the diagrams apply the correct UML diagram and notation.
Decision Making	20%	Sections 2.5 to 7.5 give each design decision with its rationale.
Clarity of Response	15%	Tables, one landscape diagram per scenario, and a final check in Sections 2.7 to 7.7.

References

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language user guide* (2nd ed.). Addison-Wesley.

Clements, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., Merson, P., Nord, R., & Stafford, J. (2010). *Documenting software architectures: Views and beyond* (2nd ed.). Addison-Wesley.

Cockburn, A. (2001). *Writing effective use cases*. Addison-Wesley.

Fowler, M. (2003). *UML distilled: A brief guide to the standard object modeling language* (3rd ed.). Addison-Wesley.

Harel, D. (1987). Statecharts: A visual formalism for complex systems. *Science of Computer Programming, 8*(3), 231–274.

Jacobson, I., Christerson, M., Jonsson, P., & Övergaard, G. (1992). *Object-oriented software engineering: A use case driven approach*. Addison-Wesley.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1